ASSIGNEMENT 01

MAX MARKS: 05 LAST DATE: 8-SEPTEMBER-2025

Assignment Submission Guidelines

1. You are required to **submit both a handwritten assignment and its PDF copy**.
2. Bring your **handwritten work** to the **first class of next week**.
3. Marks will only be awarded based on your performance in the viva.
4. Ensure **timely submission** — **late submissions will not be accepted** under any circumstances.
5. The submitted file must include **clear images of your handwritten work**.
6. Follow the required **file naming format** strictly:

   ROLL_NO_NAME_DISCIPLINE_SECTION.pdf

ANALYSIS OF RECURSIVE ALGORITHMS

QUESTION 01 :-
Factorial Calculation: Compute the factorial of a number *n* (n!).

- Focus: Recurrence relation is fact(n) = n * fact(n-1), base case fact(0) = 1.

```
def factorial(n):
    if n == 0:  # Base case
        return 1
    else:
        return n * factorial(n - 1)  # Recursive call
```

QUESTION 02 :-
Fibonacci Sequence: Generate the *n*-th Fibonacci number.

- Focus: Recurrence relation is fib(n) = fib(n-1) + fib(n-2), base cases fib(0) = 0, fib(1) = 1

```python
def fibonacci(n):
    if n == 0:  # Base case
        return 0
    elif n == 1:  # Base case
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)
```

QUESTION 03:-

```python
def fun(x):

    if(x > 0):
        x -= 1
        fun(x)
        print(x , end=" ")
        x -= 1
        fun(x)

# Driver code
a = 4
fun(a)
```

QUESTION 04:-

Permutations of a String: Generate all possible permutations of a string of length *n*.

- **Focus**: Recursively generate permutations by selecting each character as the first and permuting the rest; base case is a string of length 1.

```python
def permutations(s):
    if len(s) <= 1:  # Base case: Single character or empty
        return [s]
    result = []
    for i in range(len(s)):
        current = s[i]
        remaining = s[:i] + s[i+1:]  # Exclude current character
        for p in permutations(remaining):  # Recursive call
            result.append(current + p)
    return result
```

ANALYSIS OF NON RECURSIVE ALGORITHMS

QUESTION 05:-

Sum of Array Elements: Compute the sum of all elements in an array of *n* numbers.

- **Focus**: Single loop iterating through the array to accumulate the sum.

```
def array_sum(arr):
    total = 0
    for i in range(len(arr)):  # Loop through each element
        total += arr[i]
    return total
```

QUESTION 06:-

Reverse a String: Reverse a string of length *n* in-place using two pointers.

Focus: Loop using two pointers (start and end) swapping characters until they meet.

```
def reverse_string(s):
    s = list(s)  # Convert to list for mutability
    left, right = 0, len(s) - 1
    while left < right:  # Loop until pointers meet
        s[left], s[right] = s[right], s[left]  # Swap
        left += 1
        right -= 1
    return ''.join(s)
```

QUESTION 07:-

Power Calculation: Compute *a* raised to the power *n* (a^n) using iterative multiplication.

- **Focus**: Loop to multiply the base *a* by itself *n* times, optimized using exponentiation by squaring.7

```
def power(a, n):
    result = 1
    base = a
    while n > 0:  # Loop until exponent is 0
        if n % 2 == 1:  # If n is odd, multiply result by base
            result *= base
        base *= base  # Square the base
        n //= 2  # Divide n by 2
    return result
```

QUESTION 08:-

Queue-based Level Order Traversal of Binary Tree: Traverse a binary tree level by level using a queue.

- **Focus**: Use a queue to process nodes level by level; assume a tree with *n* nodes.

```
from collections import deque

class TreeNode:
    def __init__(self, value=0, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right

def level_order_traversal(root):
    if not root:
        return
    queue = deque([root])  # Initialize queue with root
    while queue:  # Loop until queue is empty
        node = queue.popleft()  # Dequeue node
        print(node.value)  # Process node
        if node.left:
            queue.append(node.left)  # Enqueue left child
        if node.right:
            queue.append(node.right)  # Enqueue right child
```

Name:- Muhammad Obaid
Roll. No:- CT-23025
Class:- CSIT-A

DAA Assignment 01

Analysis Of Recursive Algorithms:-

Q1

i) Recurrence Relation:-

$$factorial(n) = \begin{cases} 1 & n = 0 \\ n * factorial(n-1) & n > 0 \end{cases}$$

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

ii) Substitution Analysis:-

$$\because T(n) = T(n-1) + 1 \quad \text{--- (a)}$$

Replace $n = n-1$

$$T(n-1) = T(n-2) + 1$$

Now substitute this in eq (a)

$$T(n) = T(n-2) + 2 \quad \text{--- (b)}$$

Again Replace $n = n-2$ in eq (a)

$$\Rightarrow T(n-2) = T(n-3) + 1$$

Now replace above eq in eq (b)

$$T(n) = T(n-3) + 3$$

Assume it will run 'k' times:-

$$\cancel{T(k) =}$$

$$T(n) = T(n-k) + k$$

If $n-k = 0 \Rightarrow n = k$

So,

$$T(n) = T(n-n) + n$$

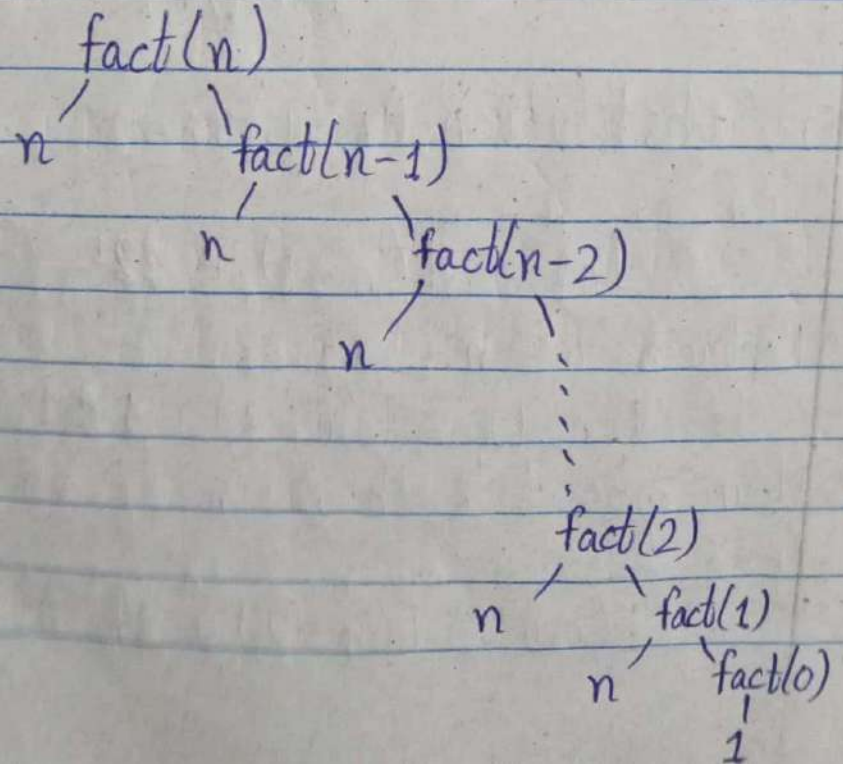
$$T(n) = T(0) + n \qquad \cancel{\because n \approx 2n}$$

$$T(n) = n + 1 \qquad \because T(0) = 1$$

$$T(n) = O(n)$$

So the time complexity is $O(n)$

iii) Recursive Tree



Since at each level, there is a constant amount of work [$O(1)$] for multiplication until the recursion reaches base case fact(0). Also since height of tree is 'n' so;

$$T(n) = O(1) + 1 + 1 \ldots + 1 \quad (n\text{-times})$$
$$T(n) = O(n)$$

Q2

i) Recurrence Relation:-

$$fib(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ fib(n-1) + fib(n-2) & n > 1 \end{cases}$$

$$T(n) = \begin{cases} 1 & n = \{0, 1\} \\ T(n-1) + T(n-2) + 1 & n > 1 \end{cases}$$

↳ For addition

ii) Substitution Analysis:-

$$\therefore T(n) = T(n-1) + T(n-2) + 1 \quad -(a)$$

Replacing $n = n - 1$

$$T(n-1) = T(n-2) + T(n-3) + 1$$

Now substitue in eq (a)

$$T(n) = T(n-2) + T(n-3) + 1 + T(n-2) + 1$$
$$T(n) = 2T(n-2) + T(n-3) + 2 \quad -(b)$$

Again Replace $n = n - 2$ in eq (a)

$$T(n-2) = T(n-3) + T(n-4) + 1$$

Now eq (b) becomes: -

$$T(n) = 2[T(n-3)+T(n-4)+1] + T(n-3) + 2$$
$$T(n) = 3T(n-3) + 2T(n-4) + 4$$

↳ For easiness, conside coefficient = 4

If it runs for 'k' times, then:-

$$T(n) = 2^0 T(n-k) + 2^1 T(n-k+1) + 2^2 T(n-k+2) + \ldots\ldots + 2^k$$

If we suppose $n-k=0 \Rightarrow n=k$, So: -

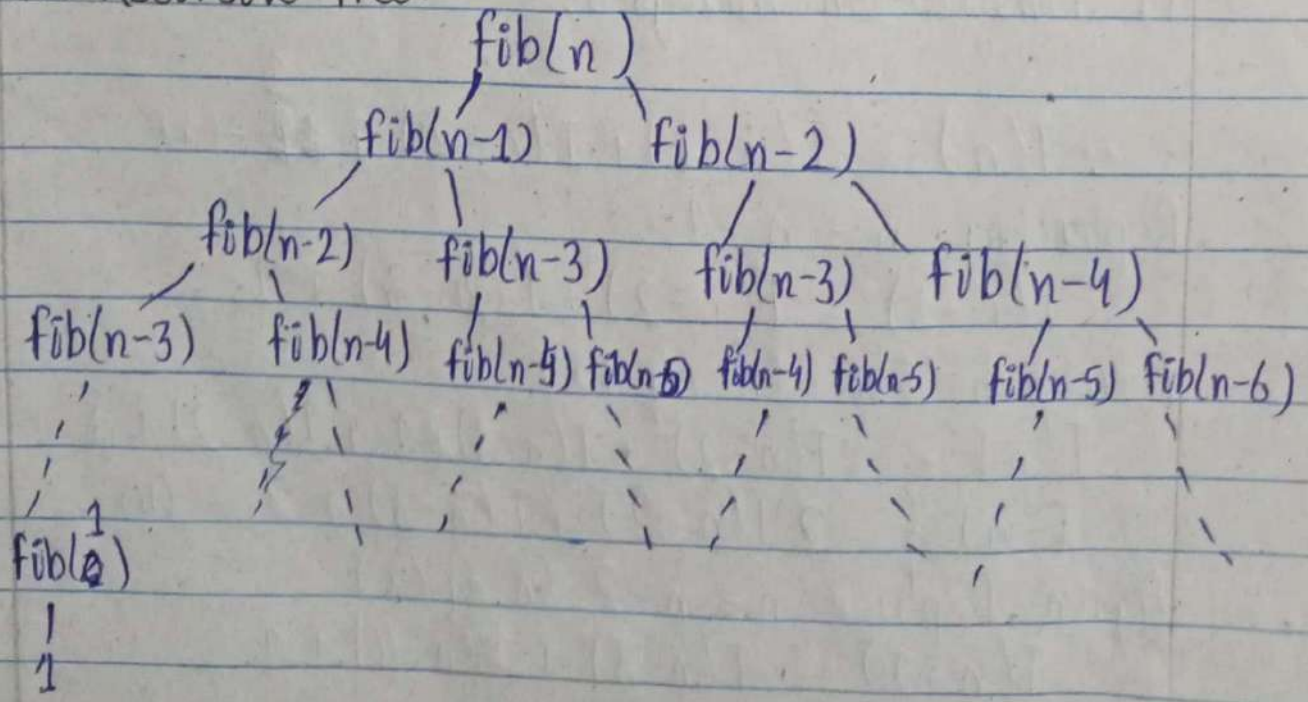
$$T(n) = 2^0 T(\cancel{n-n}\ 0) + 2^1 T(1) + 2^2 T(2) + \ldots + 2^n$$
$$= 2^{n+1} - 1 \quad \text{or} \quad \sum_{i=0}^{k} (2^i) = 2^{k+1} - 1$$

↳ 1 extra term due to constant

$$= 2 \cdot 2^n - 1$$
$$T(n) = O(2^n)$$

So the time complexity is $O(2^n)$.

iii) Recursive Tree: -



Since each child has "twice" the nodes as its parent; also the tree of height 'n' grows exponentially so;

$$T(n) = 1 + 2 + 4 + \dots + 2^{n} + 1 \text{ (of all base cases conditional returning)}$$

$$T(n) = 2^{n+1} - 1 = O(2^n)$$

Q3

i) Recurrence Relation:-

$$fun(x) = \begin{cases} \text{print}(x) \\ + fun(x-1), \\ f(x-2) \end{cases} \quad x > 0$$

$$T(x) = \begin{cases} 1 & x = 0 \\ T(x-1) + T(x-2) + 3 & x > 0 \end{cases}$$

ii) Substitution Analysis:-

$$\because T(n) = T(n-1) + T(n-2) + 3 \quad -(a)$$

Replace $n = n-1$

$$\Rightarrow T(n-1) = T(n-2) + T(n-3) + 3$$

Now eq (a) becomes:-

$$T(n) = T(n-2) + T(n-3) + 3 + T(n-2) + 3$$

$$T(n) = 2T(n-2) + T(n-3) + 6 \quad -(b)$$

Now again replace $n = n-2$ in eq (a)

$$T(n-2) = T(n-3) + T(n-4) + 3$$

Substitute $T(n-2)$ in eq (b)

$$T(n) = 2[T(n-3) + T(n-4) + 3] + T(n-3) + 6$$
$$T(n) = 3T(n-3) + 2T(n-4) + 12$$

If the algorithm runs 'k' times, then:-

~~$T(n) =$~~

assume $3T(n-3) \approx 4T(n-3)$ so now.

$$T(n) = 4T(n-3) + 2T(n-4) + 12$$

If the algorithm runs 'k' times, then:-

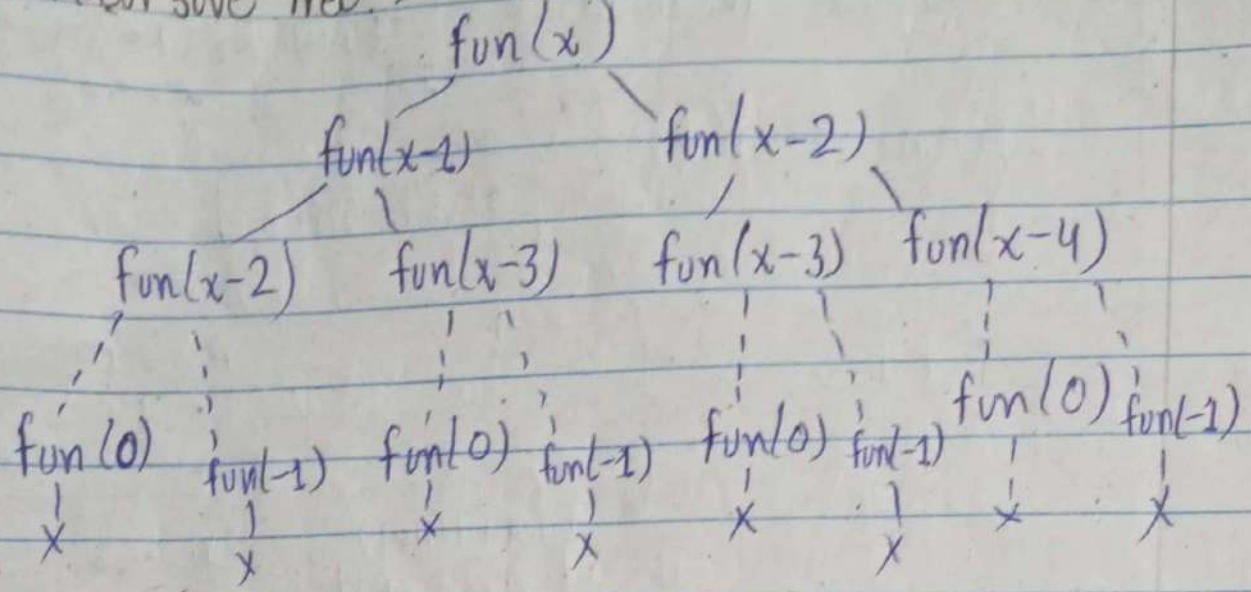
$$T(n) = 2^0 T(n-k) + 2^1 T(n-k+1) + 2^2 T(n-k+2) + \cdots + 3(2^k - 1)$$

Assume $n-k=0 \Rightarrow n=k$

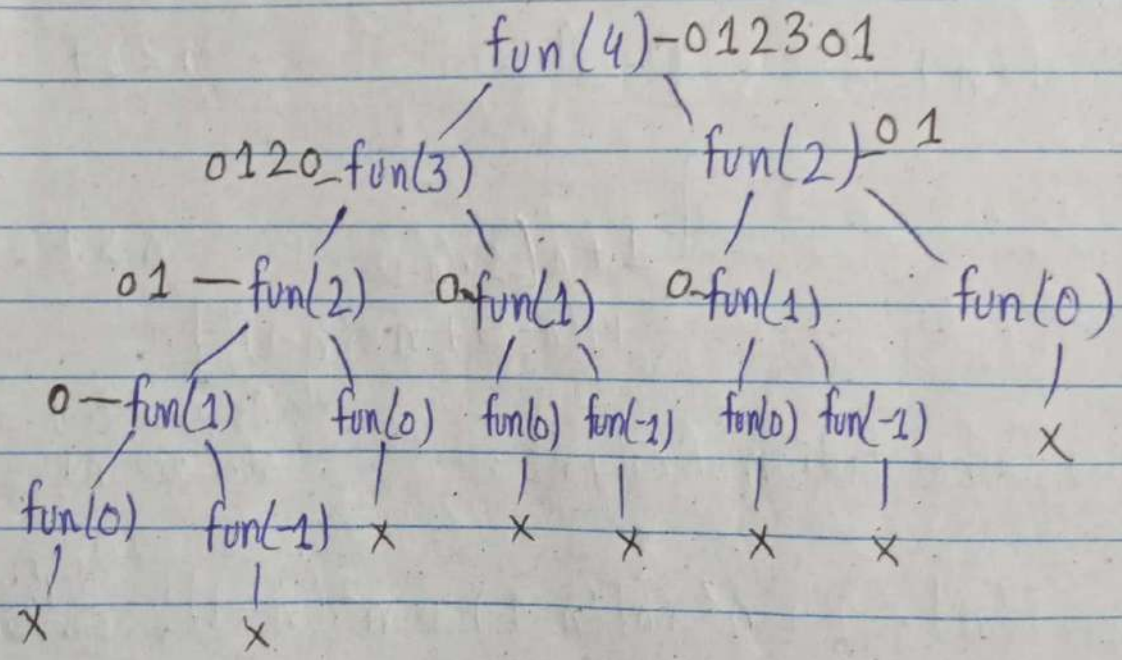
$$T(n) = 2^0 T(0) + 2^1 T(1) + 2^2 T(2) + \cdots + 3(2^k - 1)$$
$$T(n) = (2^n - 1) + 3(2^n - 1)$$
$$T(n) = O(2^n)$$

So the time complexity is $O(2^n)$.

iii) Recursive Tree:-



Dry Run for Driver Code a=4, fun(a)



Q4

i) Recurrence Relation:-

$$permutation(s) = \begin{cases} s & len <= 1 \\ s[:i] + s[i+1:] & len > 1 \end{cases}$$

$$T(n) = \begin{cases} 1 & n <= 1 \\ n + nT(n-1) & n > 1 \end{cases}$$

ii) Substitution Analysis:-

$T(n) = n + nT(n-1)$ -(a)

Replace $n = n-1$

$\Rightarrow T(n-1) = (n-1) + (n-1)T(n-2)$

Now substitute in eq (a)

$T(n) = n + n[n-1 + (n-1)T(n-2)]$

$T(n) = n + n(n-1) + n(n-1)T(n-2)$ -(b)

Now replace $n = n-2$ in eq (a)

$T(n-2) = n-2 + (n-2)T(n-3)$

Now eq (b) becomes: -

$$T(n) = n + n(n-1) + n(n-1)\left[(n-2) + (n-2)T(n-3)\right]$$

$$T(n) = n + n(n-1) + n(n-1)(n-2) + n(n-1)(n-2)T(n-3)$$

If the algorithm runs for 'k' iterations, so: -

$$T(n) = \cancel{k+k}\ n + n(n-1) + n(n-1)(n-2) + \cdots + n(n-1)(n-2)\cdots(n-k+1)T(n-k)$$

Assume: -

$$n-k = 0 \Rightarrow n = k$$

$$T(n) = n + n(n-1) + n(n-1)(n-2) + \cdots + n(n-1)(n-2)\cdots 1\ T(0)$$

$$T(n) = \frac{n!}{(n-1)!} + \frac{n!}{(n-2)!} + \cdots + \frac{n!}{2!} + 2n!$$

$$T(n) = n!\left[\frac{1}{(n-1)!} + \frac{1}{(n-2)!} + \cdots + \frac{1}{2!} + 2\right]$$

$$T(n) = n!\left[\frac{\cancel{2}}{\cancel{1}} \sum_{k=2}^{n-1} \frac{1}{k!} + \cancel{2} + 2\right]$$

$$\because e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} \Rightarrow \sum_{k=2}^{n-1} \frac{1}{k!} = e - 1 - 1$$

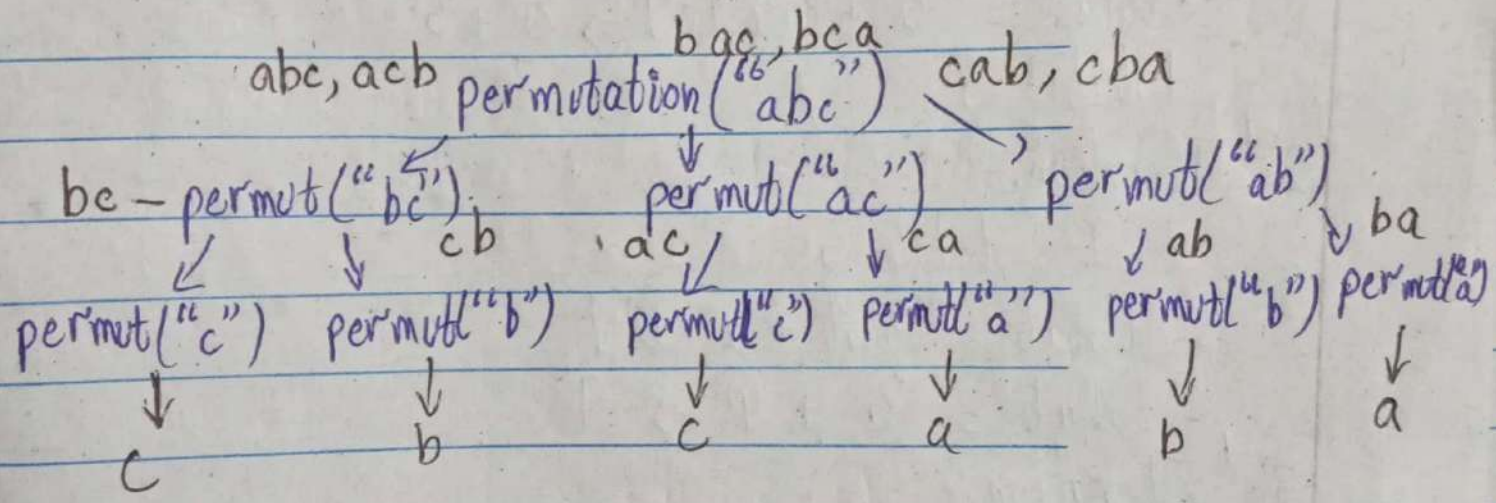
So,

$$T(n) = n!\,[e - 2 + 2] = en!$$

Also 'e' is constant so

$$T(n) = n! = O(n!)$$

iii) Recursive Tree:-



Analysis of Non Recursive Algorithms:-

Q5

i) Iterative Analysis:-

arr =

5	4	3	7
0	1	2	3

, $n = 4$

1st Iteration:-

$i = 0$, $total = 0$

$total = total + arr[i]$

$total = 0 + arr[0] \Rightarrow 0 + 5$

$total = 5$

2nd Iteration:-

$i = 1$, $total = 5$

$\Rightarrow total = total + arr[1]$

$total = 5 + 4 \Rightarrow 9$

3rd Iteration:-

$i = 2$, $total = 9$

$\Rightarrow total = total + arr[2]$

$total = 9 + 3 = 12$

4th Iteration:-

$i = 3$, $tota = 12$

$total = total + arr[i]$

$total = 12 + arr[3] = 12 + 7$

$total = 19$

5th Iteration:-

Since $i = 4 < n = 4$ ✗

so $total = 19$

ii) Time Analysis:-

a) No. of inputs = n

b) Basic Operations: Assignment, Addition, Comparison

c) $T(n)$ = No. of times basic operations execute

$$= \sum_{i=0}^{n-1} (1)$$

$$= n-1-0+1$$

$$T(n) = n \Rightarrow O(n)$$

So the time complexity is $O(n)$.

Q6

i) Iterative Analysis:-

	r	o	c	k	e	t
	0	1	2	3	4	5

s = [r o c k e t] n = 6

1st Iteration:-

left = 0, right = 5

s[left], s[right] = s[right], s[left]

s[0], s[5] = t, r (swap)

2nd Iteration:-

left = 1, right = 4

swap(s[1], s[4]) so now,

s[4] = o & s[1] = e

3rd Iteration:-

left = 2, right = 3

swap(s[2], s[3]) so now

s[3] = c, s[2] = k

4th Iteration:-

left = 3, right = 2

Since left < right ✗ ; so

s = t e k c o r

ii) Time Analysis:-

a). No. of inputs = n

b) Basic Operations:-

Comparison, Assignments, Addition, Substraction

c) T(n) = How many times operations execute

Since no. of swaps = $\left\lfloor \frac{n}{2} \right\rfloor$

So,

$$T(n) = \sum_{i=0}^{\lfloor n/2 \rfloor - 1} (1)$$

$$= \left(\frac{n}{2} - 1 - 0 + 1\right) = \frac{n}{2}$$

$$T(n) = O(n)$$

So the time complexity is $O(n)$.

Q7

i) Iterative Analysis:-

a = 3 , n = 4

1st Iteration:-

result = 1 , base = 3

$\Rightarrow$ n % 2 == 1

4 % 2 = 0 ✗

base = base * base

base = 3 * 3 = 9 ; n = n/2 = 4/2 = 2

2nd Iteration:-

result = 1 , base = 9

=> n % 2 == 1 ✗ so,

base = base * base

base = 9 * 9 = 81 ; $n = \frac{n}{2} = \frac{2}{2} = 1$

3rd Iteration:-

result = 1, base = 81

=> n % 2 == 1 ✓ then

result = result * base = 1 * 81 = 81

base = 81 * 81 = 6561 ; $n = \frac{n}{2} = \frac{1}{2} = 0$

4th Iteration:-

Since n > 0 ✗ so

result = 81

ii) Time Analysis:-

i) No. of inputs = n

ii) Basic Operations:-

Assignment, Remainder, Comparison, Multiplication, Division

iii) T(n) = No. of times basic operation execute

Since each time 'n' is reducing by half

so max-iterations = $\log_2(n) + 1$ → when n==0

$$T(n) = \sum_{i=0}^{\lfloor \log_2(n) \rfloor + 1} (1)$$

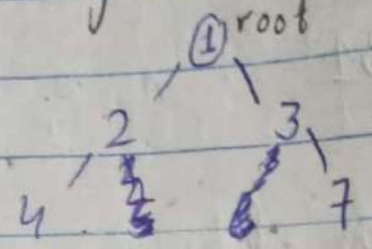
$T(n) = \log_2(n) + 1 - 0 + 1$

$T(n) = \log_2(n) + 2 = O(\log_2(n))$

Q8

i) Iterative Analysis:-

$n = 5$ - nodes



1st Iteration:-

queue = [1 | | |]

node = 1

print(1)

node.left present ✓ => queue = [2 | | |]

node.right present ✓ => queue = [2 | 3 | | |]

2nd Iteration:

node = 2

print(2)

if node.left ✓ => queue = [3 | 4 | | |]

node.right present X

3rd Iteration:-

node = 3 . print(3)

node.left X

node.right present ✓ => queue = [4 | 5 | |]

4th Iteration:-

node = 4

print(4)

node.left present X node.right present X

queue = [7 | | |]

5th Iteration:-

node = 7 , print(7)

node.left present X , node.right present X

6th Iteration:-

queue = [| | | |]

Since queue is empty so return.

ii) Time Analysis:-

i) No. of inputs = n

ii) Basic Operations:-

Assignment, Comparison, Function-call

iii) $T(n) = \sum_{i=0}^{n} (1)$

$= n - 0 + 1 = n + 1$ where n = no. of nodes

$T(n) = O(n)$

So time complexity of algorithm is $O(n)$.